Project - High Level Design

on

Real Estate Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Siddhi Suraniya	EN22CS301956
2	Shrishti Chouksey	EN22CS301935
3	Sandesh Vani	EN22CS301862
4	Shivam Choudhary	EN22CS301903
5	Shreya Gupta	EN22CS301927
6	Shyam Narayan Pandey	EN22CS301949

Group Name: Group 12D8

Project Number: AAI-47

Industry Mentor Name:

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

1.1 Scope of the document

This document defines the high-level design of a **production-grade reactive Real Estate Support Triage Agent** that leverages **Large Language Models (LLMs)** to automate and accelerate initial communication workflows. It covers the architectural components responsible for real-time text classification, urgency and intent detection, Named Entity Recognition (NER) for extracting critical data points such as identifiers and dates, and LLM-driven response generation for contextually accurate draft replies.

The scope includes system architecture, agent workflow, data flow between frontend, backend, and LLM services, and key design considerations related to scalability, reliability, and response latency. Detailed implementation logic, UI design, database schemas, and infrastructure-level configurations are outside the scope of this document.

1.2 Intended Audience

This document is intended for **technical stakeholders, system architects, backend and AI engineers, product owners, and evaluators** involved in the design, review, or integration of the Real Estate Support Triage Agent. It is also relevant for teams responsible for extending the system with CRM platforms, analytics pipelines, or additional AI capabilities. The document assumes a basic understanding of web services, APIs, and AI-driven systems.

1.3 System overview

The Real Estate Support Triage Agent is a **reactive, AI-powered communication system** designed to automate the initial handling of real estate inquiries. The system processes incoming text messages in real time, classifies them based on intent and urgency, extracts critical entities such as IDs and dates using Named Entity Recognition (NER), and generates context-aware draft responses using a Large Language Model (LLM).

The architecture follows a modular design where a frontend captures user inputs, a backend orchestrates validation and workflow control, and an AI agent layer performs reasoning, classification, and response generation. The system outputs structured data and AI-generated responses to significantly reduce response times and improve operational efficiency in real estate support workflows.

2. System Design

2.1 Application Design

The Real Estate Support Triage Agent is designed as a **reactive, event-driven application** that processes incoming real estate communications in real time. The system follows a modular architecture where responsibilities are clearly separated across the presentation layer, orchestration layer, and AI intelligence layer.

This design ensures low latency, scalability, and ease of integration with external services Incoming text requests are validated and routed by the backend service, which invokes AI pipelines for intent classification, urgency detection, and Named Entity Recognition (NER). Based on the extracted context and classification results, the system leverages a Large Language Model (LLM) to generate a structured, contextually accurate draft response. The final output is returned as a structured JSON response, ready for downstream systems or human review.

2.3 Process Flow.

1. Request Intake: Incoming real estate communication is received through the frontend or external client system and forwarded to the backend API for processing.

2. Input Validation: The backend validates the request format, checks for empty or malformed inputs, and ensures the message is suitable for AI processing.

3. Intent Classification: The system analyzes the text using an AI classification pipeline to determine the user's intent, such as inquiry, complaint, follow-up, or urgent request.

4. Urgency Detection: Based on content signals and classification confidence, the system assigns an urgency level to prioritize time-sensitive or critical requests.

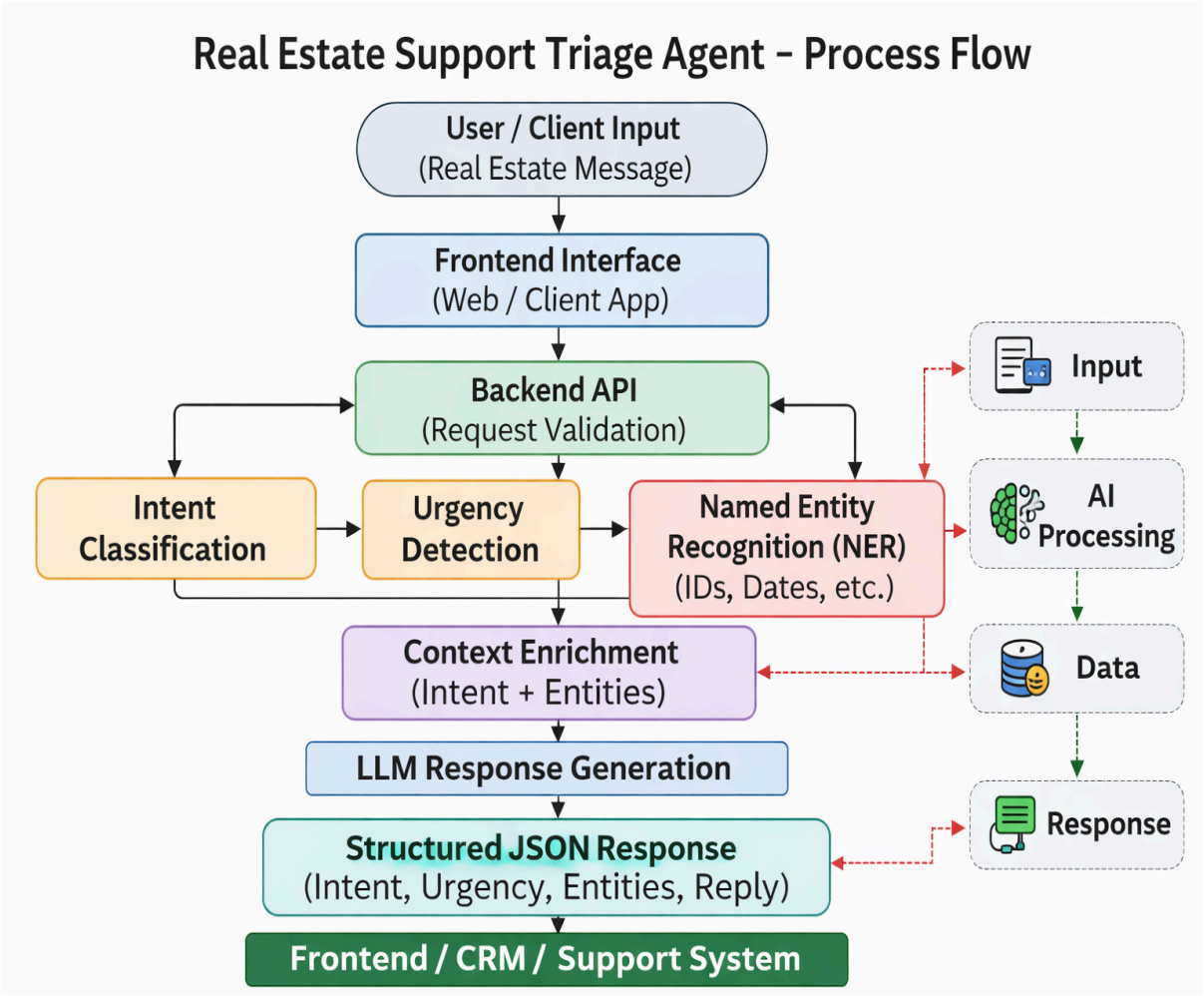
5. Named Entity Recognition (NER): Key entities such as property IDs, booking references, dates, locations, and contact-related information are extracted from the text.

6. Context Enrichment: Extracted entities and classification results are combined to build a structured context for accurate response generation.

7. LLM Response Generation: A Large Language Model is invoked to generate a context-aware draft response aligned with the detected intent and urgency.

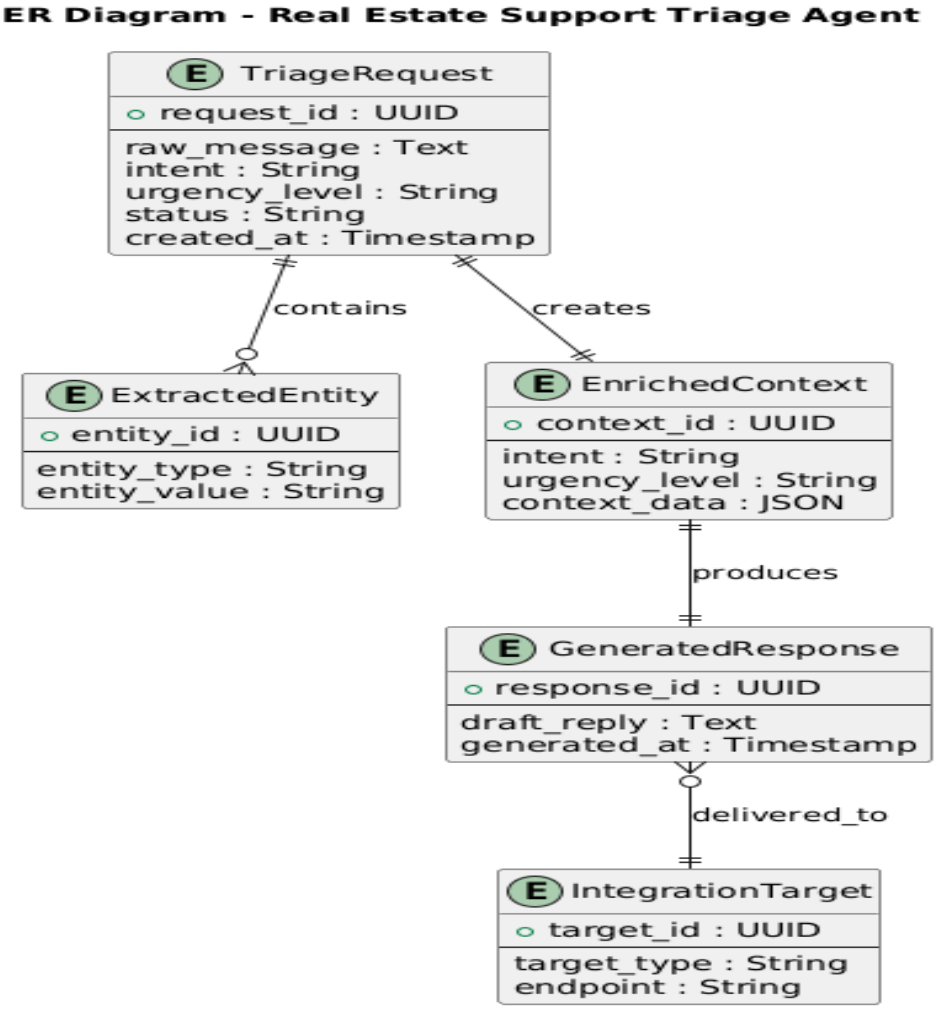
8. Response Structuring: The generated output is formatted into a structured JSON response, including intent, urgency, extracted entities, and suggested reply.

9. Response Delivery: The structured response is returned to the frontend or integrated support system for display, review, or further action.



2.3 Information Flow

1. **Input Reception:** User or client submits a real estate message through the frontend or integrated communication channel.
2. **API Transmission:** The message is transmitted to the backend API layer in a standardized request format.
3. **AI Processing Pipeline:** The backend forwards the input to the AI pipeline for intent
4. classification, urgency detection, and Named Entity Recognition (NER).
5. **Context Aggregation:** Extracted entities and classification results are combined to form a structured contextual representation.
6. **LLM Interaction:** The enriched context is sent to the LLM API to generate a contextually accurate draft response.
7. **Response Structuring:** The generated output is formatted into a structured JSON object containing intent, urgency, extracted entities, and suggested reply.
8. **Response Distribution:** The structured response is delivered back to the frontend or downstream systems such as CRM or support dashboards.



2.4 Components Design

The system is composed of modular components designed to support scalability, low latency, and clear separation of concerns. The frontend interface serves as the interaction layer, responsible for capturing real estate messages from users or client systems and displaying the generated responses. The backend API layer acts as the central orchestrator, handling request validation, workflow routing, and coordination between AI services and downstream integrations.

Core intelligence is implemented through dedicated AI modules, including intent and urgency classification, Named Entity Recognition (NER), and context enrichment. These components analyze incoming text, extract critical domain-specific information such as identifiers and dates, and construct a structured context for response generation. The Large Language Model (LLM) response generation module produces contextually accurate draft replies, which are then formatted by the response structuring module into standardized JSON outputs. An integration layer enables seamless delivery of these structured responses to frontend applications, CRM systems, or support dashboards for further action or review.

2.5 Key Design Considerations

- **Real-Time Responsiveness:** The system is designed to process incoming messages with minimal latency to ensure timely triage and rapid response generation.
- **Modular Architecture:** Components are decoupled to allow independent scaling, maintenance, and future enhancements without impacting the entire system.
- **Accuracy of Classification and NER:** High precision in intent detection, urgency classification, and entity extraction is critical to ensure reliable triage outcomes.
- **Scalability:** The architecture supports horizontal scaling to handle increasing volumes of real estate inquiries and concurrent users.
- **LLM Integration Reliability:** The system includes controlled LLM invocation to ensure consistent output quality, error handling, and fallback mechanisms.
- **Structured Output Format:** Responses are generated in standardized JSON format to enable seamless integration with CRMs and support tools.
- **Security and Data Privacy:** Sensitive user data is handled securely with validation, access control, and secure API communication.
- **Extensibility:** The design allows easy addition of new intents, entities, communication channels, and downstream integrations.

2.6 API Catalogue.

1. Triage Request API

Purpose: Accepts incoming real estate communication for processing.
Method: `POST`
Endpoint: `/triage`
Description: Receives raw text input from frontend or external systems and initiates the triage workflow including validation, classification, NER, and response generation.

2. Intent & Urgency Classification API

Purpose: Determines the intent and urgency of the incoming message.
Method: `POST`
Endpoint: `/classify`
Description: Processes the input text and returns the detected intent (e.g., inquiry, complaint, follow-up) along with urgency level.

3. Named Entity Recognition (NER) API

Purpose: Extracts critical domain-specific entities.
Method: `POST`
Endpoint: `/extract-entities`
Description: Identifies entities such as property IDs, booking references, dates, locations, and other relevant identifiers from the input text.

4. Context Enrichment API

Purpose: Aggregates classification and entity extraction results.
Method: `POST`
Endpoint: `/enrich-context`
Description: Combines intent, urgency, and extracted entities into a structured context object for accurate response generation.

5. LLM Response Generation API

Purpose: Generates a context-aware draft response.
Method: `POST`
Endpoint: `/generate-response`
Description: Invokes the Large Language Model (LLM) API using enriched context to produce a relevant and accurate draft reply.

6. Structured Response API

Purpose: Returns the final structured output.
Method: `POST`
Endpoint: `/response`
Description: Formats and delivers the AI-generated output as a standardized JSON response containing intent, urgency, extracted entities, and suggested reply.

7. Health Check API

Purpose: Monitors service availability.
Method: `GET`
Endpoint: `/health`
Description: Verifies backend and AI service readiness for production monitoring and deployment validation.

8. Integration API (Optional)

Purpose: Sends triage results to external systems.
Method: `POST`
Endpoint: `/integrations/crm`
Description: Pushes structured triage responses to CRM platforms or support dashboards for further action

3. Data Design.

3.1. Data Model

The core data model represents a triage request lifecycle and includes the following key elements:

- Request ID
  Unique identifier assigned to each incoming real estate communication.
- Raw Message
  Original text content submitted by the user or client system.
- Intent
  Classified intent of the message (e.g., inquiry, complaint, follow-up).
- Urgency Level
  Priority assigned to the request based on content analysis.
- Extracted Entities
  Structured key-value pairs representing domain-specific entities such as property IDs, dates, locations, and references.
- Enriched Context
  Aggregated object combining intent, urgency, and extracted entities for AI processing.

- Generated Response
  Context-aware draft reply produced by the LLM.
- Timestamp
  Time at which the request was processed.
- Processing Status
  Indicates whether the request is processed, pending, or failed.

This data model ensures clarity, consistency, and extensibility, enabling accurate triage decisions and efficient downstream consumption.

3.2. Data Access Mechanism

The Real Estate Support Triage Agent follows a **controlled and service-oriented data access mechanism** to ensure consistency, security, and performance. All data access operations are handled exclusively through the backend API layer, preventing direct access from frontend or external systems. This approach enforces validation, authorization, and standardized data handling across the application.

Data related to incoming requests, classification results, extracted entities, and generated responses is accessed through well-defined API interfaces and internal service modules. Read and write operations are optimized for real-time processing, with minimal persistence for auditability and monitoring. The system supports secure interaction with external services, including LLM APIs and CRM integrations, using authenticated and encrypted communication channels.

3.3. Data Retention Policies

- Incoming real estate messages are retained only for the duration required to complete triage processing.
- Transient data used for intent classification, urgency detection, and NER is cleared after response generation.
- Minimal metadata such as request ID, timestamp, intent, urgency level, and processing status may be retained for monitoring and audit purposes.
- AI-generated draft responses are not permanently stored unless required for operational or compliance needs.
- Sensitive user content is not retained beyond the configured retention period.
- Retained data is protected through access control and secure storage mechanisms.
- Data retention duration can be configured based on business and regulatory requirements.

3.4. Data Migration

- The system is designed to operate primarily on real-time and transient data, minimizing the need for complex data migration.
- Data migration is required only when integrating with external systems such as CRM platforms or legacy support databases.
- Historical triage metadata may be migrated to enable analytics, reporting, or continuity of support operations.
- Migration processes follow structured data formats to maintain consistency in intent, urgency, and entity fields.
- Data validation is performed during migration to ensure integrity and completeness.
- Sensitive data is migrated in compliance with security and privacy policies.
- Migration activities are planned to avoid disruption to live triage operations.

4. Interfaces

- **User Interface (UI)**
  Provides an interface for users or support agents to submit real estate messages and view AI-generated triage responses.
- **Frontend–Backend Interface**
  Enables secure communication between the frontend application and backend API using RESTful HTTP requests.
- **Backend–AI Services Interface**
  Facilitates interaction between the backend and AI modules for intent classification, urgency detection, Named Entity Recognition (NER), and LLM-based response generation.
- **LLM API Interface**
  Connects the system to external Large Language Model services for generating context-aware draft responses.
- **Integration Interface**
  Allows structured triage outputs to be shared with external systems such as CRM platforms and support dashboards.

5. State and Session Management

- The system is designed to be **stateless at the API level** to support scalability and high availability.
- Each triage request is processed independently using a unique request identifier.
- Session context is maintained only for the duration of a single request lifecycle.
- Temporary state, including classification results and extracted entities, is held in memory during processing and discarded after response generation.
- No long-lived conversational state is maintained unless explicitly required by downstream integrations.
- Frontend session handling is limited to user interaction and does not store sensitive triage data.
- Stateless design enables easy horizontal scaling and fault tolerance in production environments.

6. Caching

- Caching is used to reduce latency and improve system responsiveness for high-frequency operations.
- Frequently accessed configuration data such as intent definitions and classification rules may be cached in memory.
- Reusable AI prompt templates and system instructions are cached to avoid repeated computation.
- No sensitive user message content is cached to ensure data privacy.
- Cache entries are time-bound and invalidated automatically to maintain data freshness.
- The caching layer is designed to be optional and does not affect core triage functionality if unavailable.
- Caching supports horizontal scaling by minimizing repeated calls to external AI services.

7. Non-Functional Requirements

7.1. Security Aspects

The system ensures secure communication across all components by enforcing encrypted data transmission between the frontend, backend services, and external AI APIs. Input validation and controlled API access are implemented to prevent unauthorized usage and malicious requests. Credentials and API keys required for Large Language Model (LLM) services are securely managed on the server side and are never exposed to client applications.

Sensitive user data is processed in memory and is not persistently stored unless explicitly required for operational or compliance purposes. Logging mechanisms are designed to avoid capturing confidential content, and access to administrative or monitoring interfaces is restricted through authorization controls.

7.2. Performance Aspects

The system is designed to support **real-time processing** of incoming real estate communications with minimal response latency. Efficient request handling, lightweight data structures, and optimized AI pipeline invocation ensure fast intent classification, entity extraction, and response generation. Performance-sensitive operations are executed asynchronously where applicable to prevent blocking and maintain responsiveness under normal load conditions.

The architecture supports **horizontal scalability** to handle increased traffic and concurrent requests without performance degradation. Caching mechanisms are applied selectively to reduce redundant processing and external API calls. The stateless design of backend services enables load balancing and efficient resource utilization, ensuring consistent performance in production environments.

8. References

- RESTful API Design Principles and Best Practices
- Natural Language Processing (NLP) techniques for text classification and entity extraction
- Named Entity Recognition (NER) concepts and implementations
- Large Language Model (LLM)–based response generation methodologies
- Secure API communication using HTTPS and token-based authentication
- Scalable backend architecture design for real-time systems
- Industry best practices for AI-driven support automation systems